Kulliyyah Of Information & Communication Technology

Sem 1 2023/2024

CSCI 2304

Intelligent Systems

Section 1

Group Project Report

Message Spam Detector

Group H Members:

	Name	**Matric No.**	**Contribution**
1.	Fikri bin Hisham-muddin	2112011	100%
2.	Ammar Daniel bin Abd Rahman	2115291	100%
3.	Hariz Syahmi Bin Hairo Rose Sidi	2115575	100%

Web deployment for the project : https://checkspam.fiekzz.com/

Demo video : https://youtu.be/aYipe1FcXOA

Instructed by Amelia Ritahani binti Ismail

Table of contents

Abstract

The digital landscape has become an integral part of our daily lives, with communication playing a central role in connecting individuals, businesses, and communities. However, as online communication proliferates, so does the prevalence of unwanted messages, including spam, scams, and phishing attempts. The surge in such malicious activities poses a threat to the online experience, jeopardizing user security and trust. The implications of inaccurate message classification extend beyond individual inconvenience; they can result in compromised online security, loss of sensitive information, and erosion of trust in digital communication channels. Our intelligent message spam checker seeks to mitigate these risks, offering a robust solution for individuals, businesses, and institutions alike. In essence, our project addresses the imperative for a secure and reliable digital ecosystem, aligning with the principles of Goal 16. By harnessing the power of AI, we aim to fortify online spaces, promote responsible digital interactions, and contribute to the overarching objective of fostering peace, justice, and strong institutions in the digital realm.

Introduction

In contemporary society, the proliferation of digital communication has brought about unprecedented challenges, one of which is the alarming rise in spam and phishing messages. Recognizing the need to safeguard individuals from falling victim to these deceptive tactics, we have developed an advanced "Message Spam Checker" system. The prevalence of spam messages has escalated due to the ever-expanding digital landscape, making it imperative to address this issue with sophisticated technological solutions. Our system aims to empower users by offering a reliable and efficient mechanism to discern between legitimate and malicious messages.

To achieve this, we have harnessed the power of artificial intelligence (AI) and machine learning techniques, creating a user-friendly website that incorporates the latest advancements in AI training. By leveraging datasets sourced from Kaggle, comprising a diverse array of spam and non-spam messages collected from various platforms, we ensure our model is robust and reflective of real-world scam scenarios. The primary objective of our project is twofold: firstly, to implement a system capable of accurately identifying spam messages, thereby preventing unsuspecting individuals from falling prey to phishing scams.

Secondly, we seek to raise public awareness about the pervasive threat of scams in today's digital landscape.

Through rigorous training and testing processes, we will evaluate and select the most effective machine learning algorithm based on metrics such as precision and accuracy. This careful selection process ensures that our "Message Spam Checker" not only meets the highest standards of performance but also aligns with our overarching goal of enhancing digital security. In essence, our project is not just about building a technological solution; it's about creating a platform that educates and empowers users to navigate the online space safely. By fostering awareness of the prevalent scam techniques and providing tools to combat them, we contribute to a safer and more secure digital environment for everyone.

Literature Review

Spam messages have become a prevalent issue in various online platforms, including social media networks, emails, and SMS. Detecting and filtering out spam messages is essential to ensure user privacy, security, and overall user experience. Traditional rule-based approaches to spam detection often fall short in identifying novel spam patterns, leading to the need for more advanced techniques. This literature review aims to explore the use of machine learning algorithms and techniques in creating an effective message spam detector.

Overview of Research Findings

1. Survey of review spam detection using machine learning techniques
   Crawford et al. (2015) provide a comprehensive survey of review spam detection techniques using machine learning. The study discusses various machine learning algorithms and features used for review spam detection, highlighting their effectiveness and limitations in different scenarios. This research finding emphasizes the importance of machine learning in spam detection and sets the foundation for further exploration.
2. Spam Detection on Twitter Using Traditional Classifiers
   McCord and Chuah (2011) propose a spam detection approach specifically tailored for Twitter. The study explores the use of traditional classifiers, such as Naive Bayes and Support Vector Machines, in identifying spam accounts on Twitter. This research finding demonstrates the applicability of machine learning techniques in detecting spam on specific social media platforms.

3. Stock market prediction using machine learning classifiers and social media, news
   Khan et al. (2020) discuss the use of machine learning classifiers and social media data for stock market prediction. While not directly related to message spam detection, this research finding highlights the significance of utilizing machine learning algorithms to analyze and extract insights from social media data. This insight can be leveraged in developing a message spam detector that considers contextual information from social media platforms.
4. Performance Comparison and Current Challenges of Using Machine Learning Techniques in Cybersecurity
   Shaukat et al. (2020) provide a performance comparison and identify current challenges in using machine learning techniques in cybersecurity. Although the focus is on cybersecurity, the research finding emphasizes the challenges and limitations of applying machine learning algorithms to detect malicious activities in online environments. These challenges can be extended to message spam detection and suggest potential areas of improvement.
5. Sybil Defense Techniques in Online Social Networks: A Survey
   Al-Qurishi et al. (2017) conduct a survey on Sybil defense techniques in online social networks. Although the primary focus is on countering Sybil attacks, the research finding sheds light on the challenges of identifying and deactivating fake accounts that propagate spam and fraud. This finding emphasizes the need for advanced techniques, such as machine learning, to distinguish between genuine and malicious accounts.
6. Detecting and Addressing Frustration in a Serious Game for Military Training
   DeFalco et al. (2017) investigate the detection and mitigation of frustration in a serious game for military training. While not directly related to message spam detection, this research finding highlights the importance of understanding user behavior and emotions in designing effective detection systems. Similar approaches can be applied to analyze user sentiments in messages to identify potential spam.
7. Development of an Automatic Detector of Cracks in Concrete Using Machine Learning
   Yokoyama and Matsumoto (2017) propose a system called FBS-Radar for detecting fake base stations (FBSes) in cellular networks. Although the primary focus is on combating SMS message spam and fraud, this research finding demonstrates the effectiveness of machine learning algorithms in identifying and geolocating malicious

entities. The insights from this study can be applied to message spam detection in various online platforms.

8. If it looks like a spammer and behaves like a spammer, it must be a spammer: analysis and detection of microblogging spam accounts
   Almaatouq et al. (2016) investigate the analysis and detection of microblogging spam accounts, primarily on major social media sites like Twitter, Facebook, and Quora. The research finding highlights the effectiveness of sentiment analysis and deep learning methods in classifying tweets as "spam" or "ham." This approach can be utilized in developing a message spam detector that considers user sentiment and various classifiers.
9. Real-Time Twitter Spam Detection and Sentiment Analysis using Machine Learning and Deep Learning Techniques
   Rodrigues et al. (2022) propose a real-time spam detection and sentiment analysis system for Twitter. The research finding emphasizes the use of machine learning and deep learning techniques to efficiently detect spam and analyze user sentiments. This finding contributes to the understanding of utilizing various classifiers and deep learning methods in message spam detection.
10. Anti-spam techniques based on Artificial Immune System (AIS)
    Rodrigues et al. (2022) discuss anti-spam techniques based on Artificial Immune System (AIS) for filtering SMS spam messages. The research finding highlights the utilization of features such as phone numbers, spam words, and detectors to classify messages as spam or ham. This approach can be considered in the development of a comprehensive message spam detector that incorporates AIS-based techniques.
11. A Local-Concentration-Based Feature Extraction Approach for Spam Filtering
    Zhu and Tan (2011) propose a local concentration (LC)-based feature extraction approach for spam filtering. This research finding suggests that the LC approach, combined with term selection methods and a sliding window, improves spam filtering performance compared to traditional approaches. This approach can be utilized to enhance the feature extraction process in message spam detection algorithms.
12. An Efficient Spam Detection Technique for IoT Devices Using Machine Learning
    Makkar et al. (2021) propose an efficient spam detection technique for IoT devices using graph convolutional neural networks (GCNN) and machine learning. The research finding demonstrates that leveraging both node features and the structure of the social graph improves spam bot detection in online social networks. This approach

can be extended to message spam detection in various online platforms, including social media and emails.

13. Proposed efficient algorithm to filter spam using machine learning techniques
    Aski and Sourati (2016) propose an efficient algorithm to filter spam in emails using machine learning techniques. The research finding suggests the use of pretrained bidirectional encoder representation from transformer (BERT) and machine learning classifiers to accurately classify email texts as ham or spam. This approach can be applied to message spam detection in emails, considering the contextual information within the messages.
14. Understanding Real-World Concurrency Bugs in Go
    Tu et al. (2019) propose a framework for spam detection in IoT using machine learning models. The research finding emphasizes the evaluation of different machine learning models based on input features and the computation of a spam score for IoT devices. This framework can be adapted to develop a message spam detector for IoT devices, considering the unique characteristics of IoT environments.

Knowledge Gaps and Future Research Directions

Despite the significant advancements in machine learning-based message spam detection, several knowledge gaps and potential future research directions can be identified:

1. Contextual Analysis: While sentiment analysis has been explored in identifying spam, further research is needed to incorporate more nuanced contextual analysis. This includes considering user behavior, network structure, and content semantics to improve spam detection accuracy.
2. Dynamic Adaptation: Message spam detection algorithms should be able to adapt dynamically to evolving spam patterns and techniques. Research on developing adaptive machine learning models and algorithms that can continuously learn and update spam detection rules is essential.
3. Multimodal Analysis: Incorporating multimodal data, such as images, videos, and audio, into message spam detection algorithms can enhance their effectiveness. Future research should explore the integration of machine learning techniques for analyzing and classifying multimodal content in spam detection.
4. Privacy Preservation: Developing machine learning-based spam detection techniques that prioritize user privacy is crucial. Research on privacy-preserving algorithms, such

as federated learning and differential privacy, should be explored to ensure user data is protected during the spam detection process.

5. Cross-Platform Spam Detection: While the research findings primarily focus on specific platforms such as social media, emails, and SMS, future research should investigate the development of cross-platform message spam detection algorithms. This will enable the detection and prevention of spam across multiple online platforms simultaneously.
6. Explainability and Interpretability: Machine learning models used in message spam detection should provide explainable and interpretable results to gain user trust. Research on developing interpretable machine learning models and techniques for explaining the decision-making process of spam detection algorithms is necessary.
7. Benchmark Datasets: The availability of benchmark datasets specifically designed for message spam detection can facilitate the development and evaluation of new algorithms. Future research should focus on creating standardized and diverse datasets to enable fair comparison and benchmarking of machine learning-based spam detection techniques.
8. Real-Time Detection: Real-time message spam detection is crucial to prevent spam messages from reaching users. Future research should explore the development of efficient and scalable machine learning algorithms that can perform real-time spam detection without compromising accuracy.
9. User Feedback Integration: Incorporating user feedback and preferences into the spam detection process can improve the accuracy and customization of spam filtering. Future research should investigate the development of interactive machine learning approaches that allow users to actively participate in the spam detection process.
10. Adversarial Attacks: As spam detection algorithms become more sophisticated, adversaries may develop techniques to evade detection. Future research should focus on developing robust and resilient spam detection algorithms that can withstand adversarial attacks.

In conclusion, the research findings reviewed in this literature review highlight the significance of machine learning in creating an effective message spam detector. The gaps identified and future research directions provide valuable insights for researchers and practitioners to continue advancing the field of message spam detection using machine learning techniques.

Methodology / Experimental Setup

We've chosen the datasets from Kaggle to be used in the training and testing of the AI for this project. The Datasets contain more than 5000 rows of text messages. Each row contains one message per row and is composed of two columns. The first column is the raw text message while the second row is the label of the message whether it is a ham or spam. Ham is for the message that is categorised as not a spam message while spam is a spam message. From the datasets used it could justify the message as a spam message or not spam message by using the supervised learning method to categorise it accordingly.

There are several algorithms we used in building this message spam checker model. The first one is the countVectorizer algorithm. In the context of a spam detection project, the CountVectorizer serves a pivotal role in feature extraction by converting raw text messages into a matrix of word counts. Each message is represented as a vector, with each element denoting the count of a specific word. This process results in a Document-Term Matrix (DTM), a sparse matrix where rows represent individual messages and columns encompass unique words across all messages. CountVectorizer is adept at handling variations in text, including different capitalizations and singular/plural forms, ensuring a consistent representation through tokenization. The DTM, characterized by its sparsity, functions as input features for machine learning models. Each word's count serves as a feature, enabling the model to learn patterns and associations between word occurrences and the target variable (spam or non-spam).

In a message spam detection project, the TF-IDF (Term Frequency-Inverse Document Frequency) vectorizer is instrumental in transforming raw text messages into numerical vectors. It assigns weights to words based on their frequency in a specific message and rarity across the entire dataset. This TF-IDF matrix serves as input for training the AI model, allowing it to discern important terms relevant to spam detection while mitigating the impact of common words. The algorithm's ability to highlight unique term importance enhances the model's understanding, contributing to the system's overall effectiveness in distinguishing spam from legitimate messages.

CountVectorizer's integration with classification models, such as Naive Bayes or Support Vector Machines, is crucial, as it transforms text data into a format conducive to

training the machine learning model effectively. We choose between 3 Naive Bayes which are the Gaussian, Multinomial and Bernoulli to train the AI. We compare it by their accuracy and precision score and see that the Multinomial Naive Bias is the best fit to used in our project with 95% accuracy and 100% precision score.MultinomialNB is based on the Naive Bayes algorithm, which assumes that the presence of a particular feature in a class is independent of other features. In the context of text classification, features are often word frequencies or counts.In text classification, each document is represented as a vector of word frequencies. The algorithm calculates the probability of a document belonging to a particular class (e.g., spam or non-spam) based on the observed word frequencies.

Python code

In [3]:
```
import numpy as np
import pandas as pd
```

In [4]:
```
df = pd.read_csv('spam.csv', encoding='latin-1')
```

In [5]:
```
df.sample(5)
```

Out[5]:

	v1	v2	Unnamed: 2	Unnamed: 3	Unnamed: 4
2704	spam	FreeMsg: Fancy a flirt? Reply DATE now & join ...	NaN	NaN	NaN
4755	ham	Don't make life too stressfull.. Always find t...	NaN	NaN	NaN
2280	ham	R Ì_ comin back for dinner?	NaN	NaN	NaN
4987	ham	Which channel:-):-):):-).	NaN	NaN	NaN
5255	ham	Ok... Sweet dreams...	NaN	NaN	NaN

In [6]:
```
df.shape
```

Out[6]:
```
(5572, 5)
```

In [7]:
```
# 1. Data Cleaning
# 2. EDA
# 3. Text Preprocessing
# 4. Model Building
# 5. Evaluation
# 6. Improvement
# 7. Website
# 8. Deployment
```

In [8]:
```
## 1. Data Cleaning
```

In [9]:
```
df.info()
```

In [9]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5572 entries, 0 to 5571
Data columns (total 5 columns):
 #   Column      Non-Null Count  Dtype 
---  ------      --------------  ----- 
 0   v1          5572 non-null   object
 1   v2          5572 non-null   object
 2   Unnamed: 2  50 non-null     object
 3   Unnamed: 3  12 non-null     object
 4   Unnamed: 4  6 non-null      object
dtypes: object(5)
memory usage: 217.8+ KB
```

In [10]:
```
df.head()
```

Out[10]:

	v1	v2	Unnamed: 2	Unnamed: 3	Unnamed: 4
0	ham	Go until jurong point, crazy.. Available only ...	NaN	NaN	NaN
1	ham	Ok lar... Joking wif u oni...	NaN	NaN	NaN
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	NaN	NaN	NaN
3	ham	U dun say so early hor... U c already then say...	NaN	NaN	NaN
4	ham	Nah I don't think he goes to usf, he lives aro...	NaN	NaN	NaN

In [11]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5572 entries, 0 to 5571
Data columns (total 5 columns):
 #   Column      Non-Null Count  Dtype 
---  ------      --------------  ----- 
 0   v1          5572 non-null   object
 1   v2          5572 non-null   object
 2   Unnamed: 2  50 non-null     object
 3   Unnamed: 3  12 non-null     object
 4   Unnamed: 4  6 non-null      object
dtypes: object(5)
memory usage: 217.8+ KB
```

In [12]:
```
# drop unnecessary columns
df.drop(columns=['Unnamed: 2', 'Unnamed: 3', 'Unnamed: 4'], inplace=True)
```

In [13]:
```
df.sample(5)
```

Out[13]:

	v1	v2
3718	spam	Thanks for your ringtone order, reference numb...
3547	ham	Single line with a big meaning::::: \Miss anyt...
4091	ham	I remain unconvinced that this isn't an elabor...
1332	ham	It's ok lar. U sleep early too... Nite...
3892	ham	Have you heard from this week?

In [14]:
```
# Rename columns
df.rename(columns={'v1': 'target', 'v2': 'text'}, inplace=True)
```

In [15]:
```
df.sample(5)
```

Out[15]:

	target	text
4858	ham	Hey, a guy I know is breathing down my neck to...
2167	ham	Yes.he have good crickiting mind
309	ham	Where are the garage keys? They aren't on the ...
4360	ham	Don't Think About \What u Have Got\" Think Abo...
663	ham	Leave it de:-). Start Prepare for next:-)..

In [16]:
```
from sklearn.preprocessing import LabelEncoder
encoder = LabelEncoder()
```

In [17]:
```
df['target'] = encoder.fit_transform(df['target'])
```

In [18]:
```
df.head()
```

Out[18]:

	target	text
0	0	Go until jurong point, crazy.. Available only ...
1	0	Ok lar... Joking wif u oni...
2	1	Free entry in 2 a wkly comp to win FA Cup fina...
3	0	U dun say so early hor... U c already then say...
4	0	Nah I don't think he goes to usf, he lives aro...

In [19]:
```
# missing values
df.isnull().sum()
```

Out[19]:
```
target    0
text      0
dtype: int64
```

In [20]:
```
# check for duplicate values
df.duplicated().sum()
```

Out[20]:
```
403
```

In [21]:
```
# remove duplicate values
df = df.drop_duplicates(keep='first')
```

In [22]:
```
df.duplicated().sum()
```

Out[22]:
```
0
```

In [23]:
```
df.shape
```

Out[23]:
```
(5169, 2)
```

In [24]:
```
## 2. EDA
```

In [25]:
```
df.head()
```

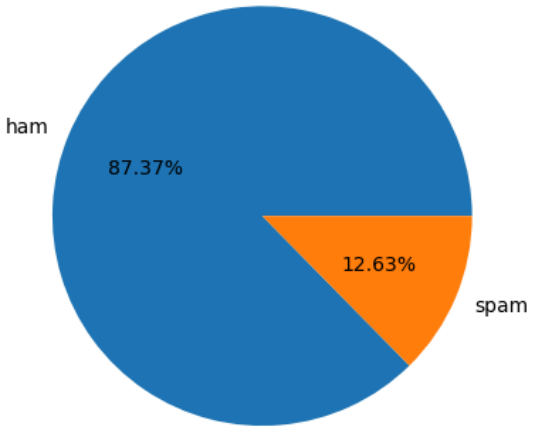
Out[25]:

	target	text
0	0	Go until jurong point, crazy.. Available only ...
1	0	Ok lar... Joking wif u oni...
2	1	Free entry in 2 a wkly comp to win FA Cup fina...
3	0	U dun say so early hor... U c already then say...
4	0	Nah I don't think he goes to usf, he lives aro...

In [26]:
```
df['target'].value_counts()
```

Out[26]:
```
target
0    4516
1     653
Name: count, dtype: int64
```

In [27]:
```
import matplotlib.pyplot as plt

plt.pie(df['target'].value_counts(), labels=['ham', 'spam'], autopct='%.2f%%')
plt.show()
```



In [28]:
```
# Data is imbalanced
```

In [29]:
```
import nltk

nltk.download('punkt')
```

```
[nltk_data] Downloading package punkt to
[nltk_data]     /Users/hidayahzainuddin/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
```

Out[29]: True

In [30]:
```
df['num_characters'] = df['text'].apply(len)
```

In [31]:
```
df.head()
```

Out[31]:

	target	text	num_characters
0	0	Go until jurong point, crazy.. Available only ...	111
1	0	Ok lar... Joking wif u oni...	29
2	1	Free entry in 2 a wkly comp to win FA Cup fina...	155
3	0	U dun say so early hor... U c already then say...	49
4	0	Nah I don't think he goes to usf, he lives aro...	61

In [32]:
```
# Fetch the numbers of words in each message
df['num_words'] = df['text'].apply(lambda x: len(nltk.word_tokenize(x)))
```

In [33]:
```
df.head()
```

Out[33]:

	target	text	num_characters	num_words
0	0	Go until jurong point, crazy.. Available only ...	111	24
1	0	Ok lar... Joking wif u oni...	29	8
2	1	Free entry in 2 a wkly comp to win FA Cup fina...	155	37
3	0	U dun say so early hor... U c already then say...	49	13

In [34]:
```
df['num_sentences'] = df['text'].apply(lambda x: len(nltk.sent_tokenize(x)))
```

In [35]:
```
df.head()
```

Out[35]:

	target	text	num_characters	num_words	num_sentences
0	0	Go until jurong point, crazy.. Available only ...	111	24	2
1	0	Ok lar... Joking wif u oni...	29	8	2
2	1	Free entry in 2 a wkly comp to win FA Cup fina...	155	37	2
3	0	U dun say so early hor... U c already then say...	49	13	1
4	0	Nah I don't think he goes to usf, he lives aro...	61	15	1

In [36]:
```
df[['num_characters', 'num_words', 'num_sentences']].describe()
```

Out[36]:

	num_characters	num_words	num_sentences
count	5169.000000	5169.000000	5169.000000
mean	78.977945	18.455794	1.965564
std	58.236293	13.324758	1.448541
min	2.000000	1.000000	1.000000
25%	36.000000	9.000000	1.000000
50%	60.000000	15.000000	1.000000
75%	117.000000	26.000000	2.000000
max	910.000000	220.000000	38.000000

In [37]:
```
# Ham messages
df[df['target'] == 0][['num_characters', 'num_words', 'num_sentences']].describe()
```

Out[37]:

	num_characters	num_words	num_sentences
count	4516.000000	4516.000000	4516.000000
mean	70.459256	17.123782	1.820195
std	56.358207	13.493970	1.383657
min	2.000000	1.000000	1.000000

50%	52.000000	13.000000	1.000000
75%	90.000000	22.000000	2.000000
max	910.000000	220.000000	38.000000

In [38]:
```
# Ham messages
df[df['target'] == 1][['num_characters', 'num_words', 'num_sentences']].describe()
```

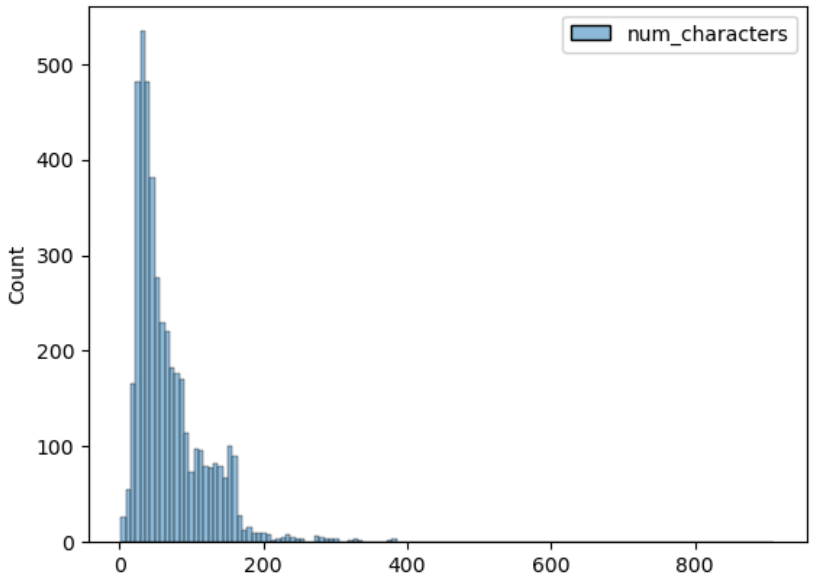
Out[38]:

	num_characters	num_words	num_sentences
count	653.000000	653.000000	653.000000
mean	137.891271	27.667688	2.970904
std	30.137753	7.008418	1.488425
min	13.000000	2.000000	1.000000
25%	132.000000	25.000000	2.000000
50%	149.000000	29.000000	3.000000
75%	157.000000	32.000000	4.000000
max	224.000000	46.000000	9.000000

In [39]:
```
import seaborn as sns
```

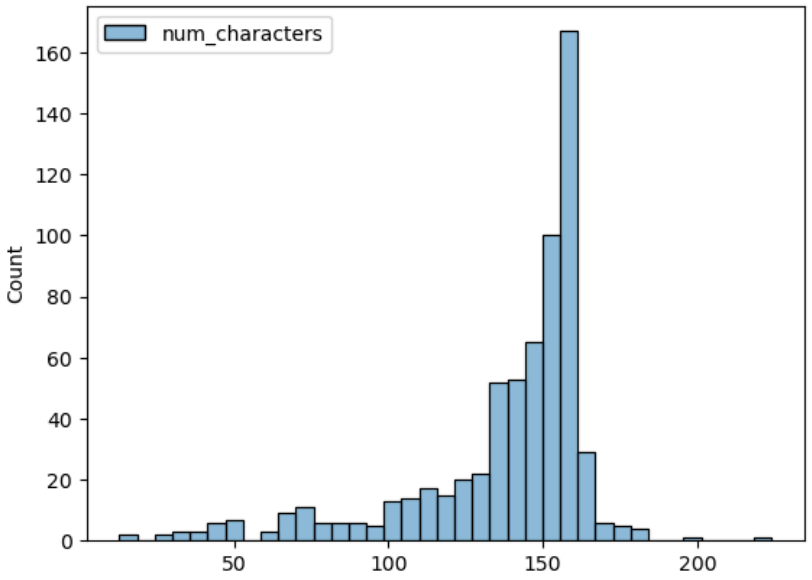
In [40]:
```
sns.histplot(df[df['target'] == 0][['num_characters']])
```

Out[40]: <Axes: ylabel='Count'>



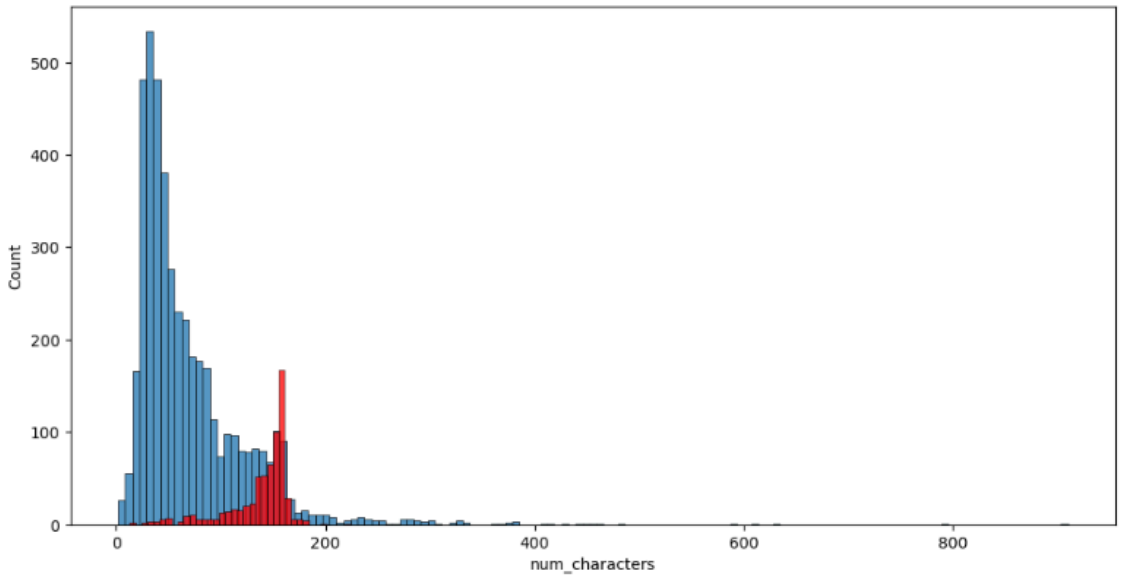
In [41]:
```
sns.histplot(df[df['target'] == 1][['num_characters']])
```

Out[41]: <Axes: ylabel='Count'>



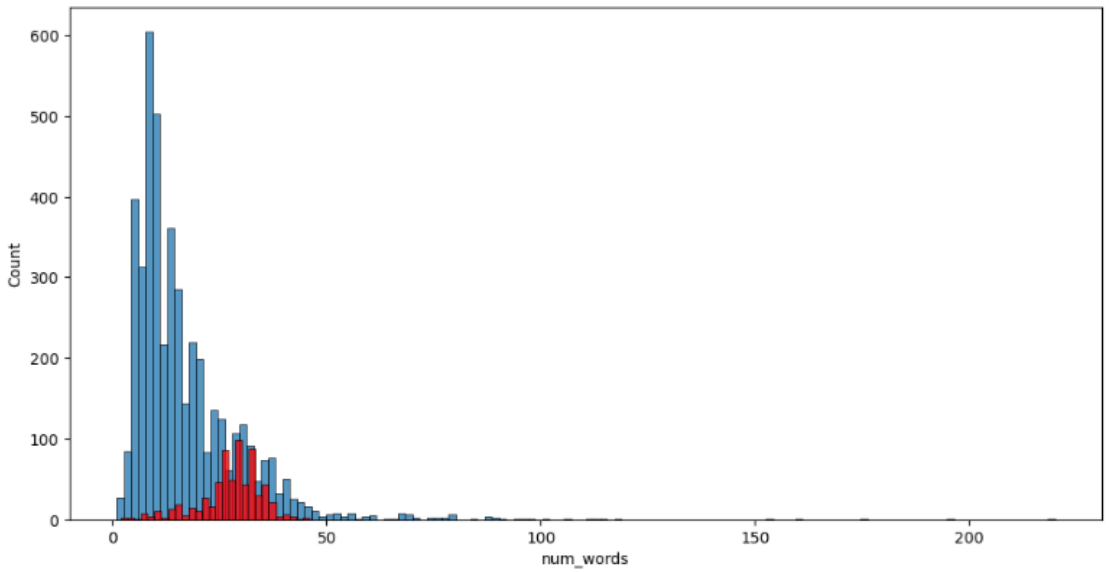
In [42]:
```
plt.figure(figsize=(12, 6))
sns.histplot(df[df['target'] == 0]['num_characters'])
sns.histplot(df[df['target'] == 1]['num_characters'], color='red')
```

Out[42]: <Axes: xlabel='num_characters', ylabel='Count'>



In [43]:
```
plt.figure(figsize=(12, 6))
sns.histplot(df[df['target'] == 0]['num_words'])
sns.histplot(df[df['target'] == 1]['num_words'], color='red')
```

Out[43]: <Axes: xlabel='num_words', ylabel='Count'>

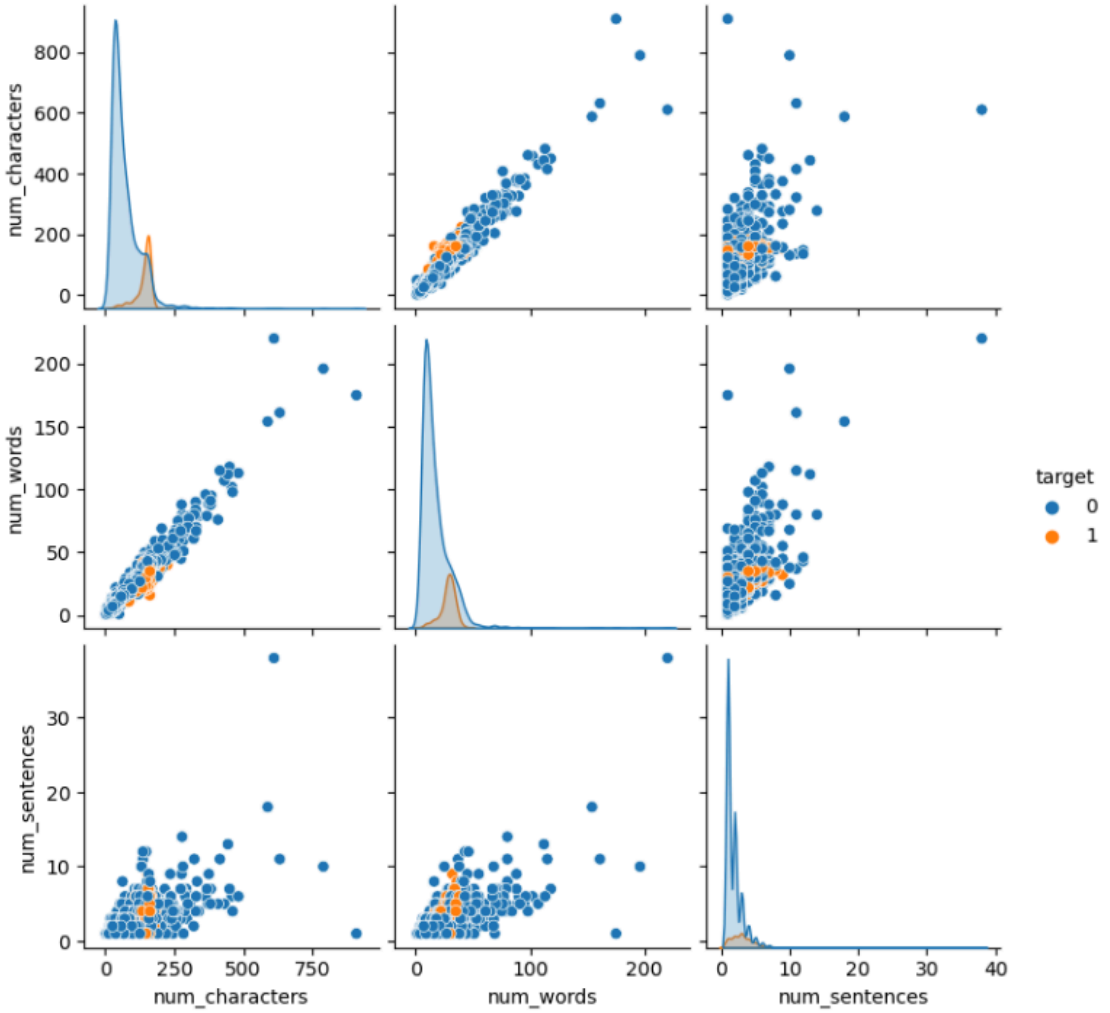


In [44]:
```
sns.pairplot(df, hue='target')
```

```
/Users/hidayahzainuddin/anaconda3/lib/python3.11/site-packages/seaborn/axisgrid.py:118: UserWarning: The fi
gure layout has changed to tight
  self._figure.tight_layout(*args, **kwargs)
```

Out[44]: <seaborn.axisgrid.PairGrid at 0x16e1aea10>

Out[44]: <seaborn.axisgrid.PairGrid at 0x16e1aea10>



In [45]:
```
df.head()
```

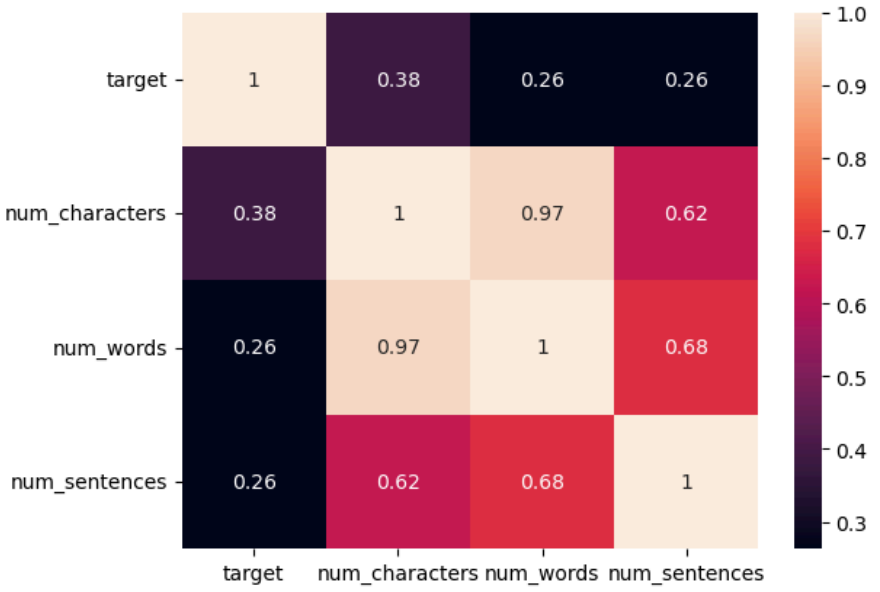
Out[45]:

	target	text	num_characters	num_words	num_sentences
0	0	Go until jurong point, crazy.. Available only ...	111	24	2
1	0	Ok lar... Joking wif u oni...	29	8	2
2	1	Free entry in 2 a wkly comp to win FA Cup fina...	155	37	2
3	0	U dun say so early hor... U c already then say...	49	13	1
4	0	Nah I don't think he goes to usf, he lives aro...	61	15	1

In [46]:

```
df1 = df.copy()
df1 = df1.drop(columns=['text'], axis=1)
sns.heatmap(df1.corr(), annot=True)
```

Out[46]: <Axes: >



In [48]:
```
import nltk
nltk.download('stopwords')

from nltk.corpus import stopwords
import string
from nltk.stem.porter import PorterStemmer
```

```
[nltk_data] Downloading package stopwords to
[nltk_data]     /Users/hidayahzainuddin/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

In [49]:
```
ps = PorterStemmer()

def transform_text(text):
    text = text.lower()
    text = nltk.word_tokenize(text)

    y = []

    for i in text:
        if i.isalnum():
            y.append(i)

    text = y[:]
    y.clear()

    for i in text:
        if i not in stopwords.words('english') and i not in string.punctuation:
            y.append(i)

    text = y[:]
    y.clear()

    for i in text:
        y.append(ps.stem(i))

    return " ".join(y)
```

In [50]:
```
transform_text("I loved the YT lectures on machine learning, How aboout you?")
```

Out[50]: 'love yt lectur machin learn aboout'

In [51]:
```
transform_text("I'm gonna be home soon and i don't want to talk about this stuff anymore tonight, k? I've
```

Out[51]: 'gon na home soon want talk stuff anymor tonight k cri enough today'

In [52]:
```
df['text'][10]
```

Out[52]:
```
"I'm gonna be home soon and i don't want to talk about this stuff anymore tonight, k? I've cried enough t
oday."
```

In [53]:
```
df['transformed_text'] = df['text'].apply(transform_text)
```

In [54]:
```
df.head()
```

Out[54]:

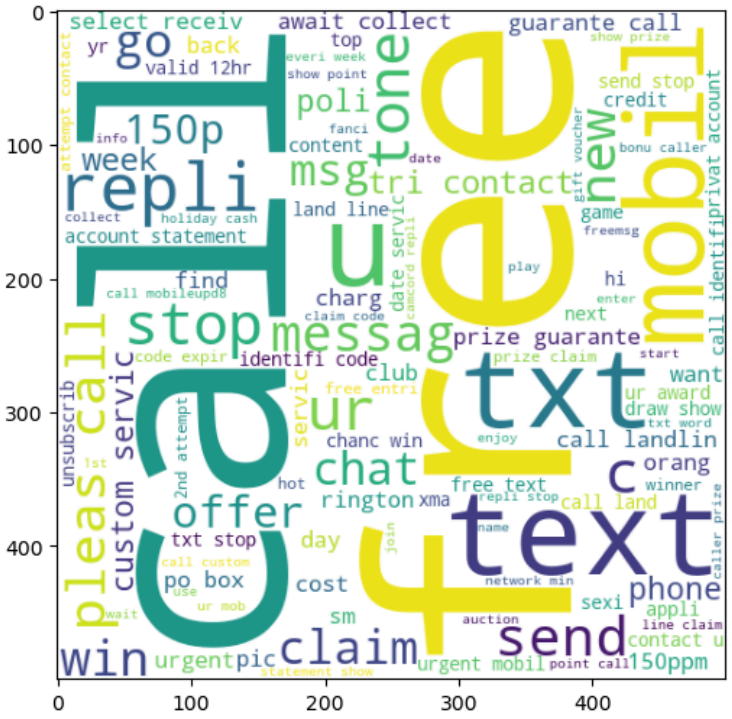
	target	text	num_characters	num_words	num_sentences	transformed_text
0	0	Go until jurong point, crazy.. Available only ...	111	24	2	go jurong point crazi avail bugi n great world...
1	0	Ok lar... Joking wif u oni...	29	8	2	ok lar joke wif u oni
2	1	Free entry in 2 a wkly comp to win FA Cup fina...	155	37	2	free entri 2 wkli comp win fa cup final tkt 21...
3	0	U dun say so early hor... U c already then say...	49	13	1	u dun say earli hor u c alreadi say
4	0	Nah I don't think he goes to usf, he lives aro...	61	15	1	nah think goe usf live around though

In [59]:
```
from wordcloud import WordCloud

wc = WordCloud(width=500, height=500, min_font_size=10, background_color='white')
```

In [60]:
```
spam_wc = wc.generate(df[df['target'] == 1]['transformed_text'].str.cat(sep=" "))
```

In [64]:
```
plt.figure(figsize=(15, 6))
plt.imshow(spam_wc)
```

Out[64]:
```
<matplotlib.image.AxesImage at 0x1778eb0d0>
```



In [65]:

```
ham_wc = wc.generate(df[df['target'] == 0]['transformed_text'].str.cat(sep=" "))

plt.figure(figsize=(15, 6))
plt.imshow(ham_wc)
```

Out[65]: <matplotlib.image.AxesImage at 0x177991410>

In [66]:
```
df.head()
```

Out[66]:

	target	text	num_characters	num_words	num_sentences	transformed_text
0	0	Go until jurong point, crazy.. Available only ...	111	24	2	go jurong point crazi avail bugi n great world...
1	0	Ok lar... Joking wif u oni...	29	8	2	ok lar joke wif u oni
2	1	Free entry in 2 a wkly comp to win FA Cup fina...	155	37	2	free entri 2 wkli comp win fa cup final tkt 21...
3	0	U dun say so early hor... U c already then say...	49	13	1	u dun say earli hor u c alreadi say
4	0	Nah I don't think he goes to usf, he lives aro...	61	15	1	nah think goe usf live around though

In [85]:
```
spam_corpus = []

for msg in df[df['target'] == 1]['transformed_text'].tolist():
    for word in msg.split():
        spam_corpus.append(word)
```

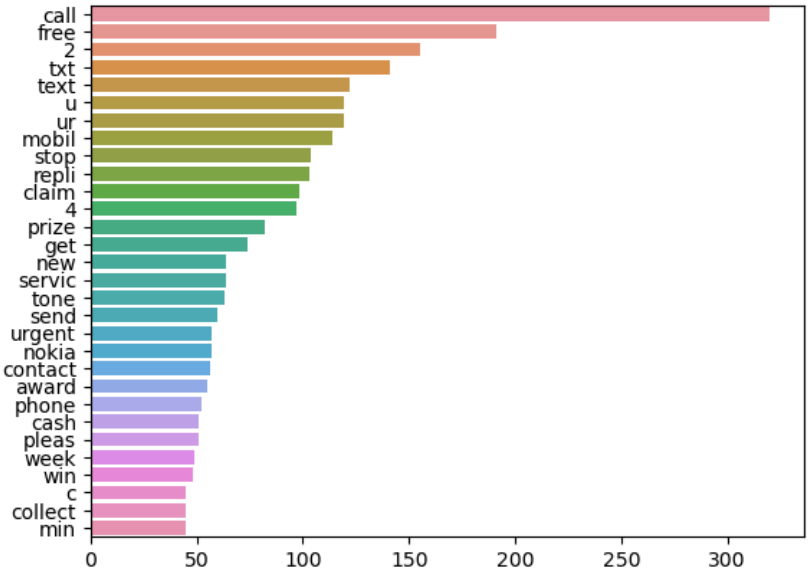
In [86]:
```
len(spam_corpus)
```

Out[86]: 9939

In [87]:
```
from collections import Counter
```

In [96]:
```
sns.barplot(y=[i[0] for i in Counter(spam_corpus).most_common(30)], x=[i[1] for i in Counter(spam_corpus)
```

Out[96]: <Axes: >



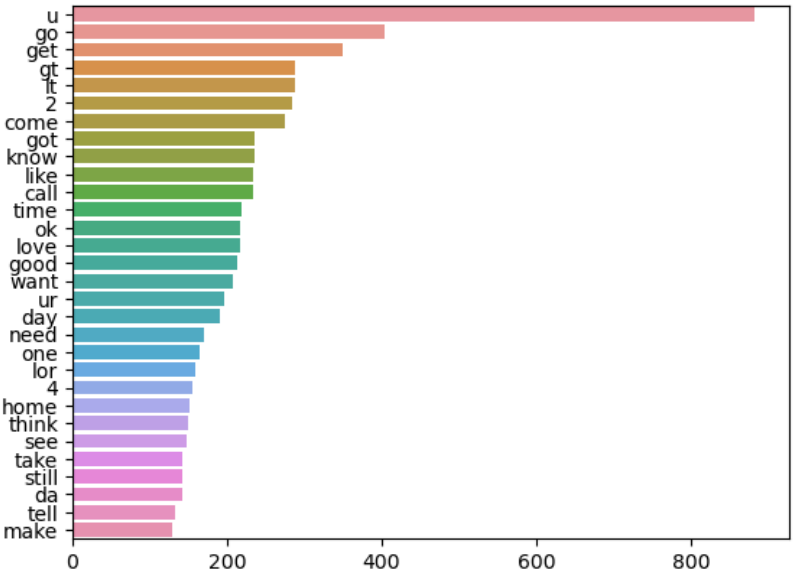
In [97]:
```
ham_corpus = []

for msg in df[df['target'] == 0]['transformed_text'].tolist():
    for word in msg.split():
        ham_corpus.append(word)
```

In [99]:
```
len(ham_corpus)
```

Out[99]: 35404

In [100…
```
sns.barplot(y=[i[0] for i in Counter(ham_corpus).most_common(30)], x=[i[1] for i in Counter(ham_corpus).m
```

Out[100… <Axes: >



In [101…
```
## 4. Model Building
```

In [117…
```
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer

cv = CountVectorizer()
tfidf = TfidfVectorizer()
```

In [118…
```
X = tfidf.fit_transform(df['transformed_text']).toarray()
```

In [119…
```
X.shape
```

Out[119… (5169, 6708)

In [120…
```
y = df['target'].values
```

In [121…
```
from sklearn.model_selection import train_test_split
```

In [122…
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=2)
```

In [123…
```
from sklearn.naive_bayes import GaussianNB, MultinomialNB, BernoulliNB
from sklearn.metrics import accuracy_score, confusion_matrix, precision_score
```

In [124…
```
gnb = GaussianNB()
mnb = MultinomialNB()
bnb = BernoulliNB()
```

In [125…
```
gnb.fit(X_train, y_train)
y_pred1 = gnb.predict(X_test)
print(accuracy_score(y_test, y_pred1))
print(confusion_matrix(y_test, y_pred1))
print(precision_score(y_test, y_pred1))
```

```
0.8762088974854932
[[793 103]
 [ 25 113]]
0.5231481481481481
```

In [126…
```
mnb.fit(X_train, y_train)
y_pred2 = mnb.predict(X_test)
print(accuracy_score(y_test, y_pred2))
print(confusion_matrix(y_test, y_pred2))
print(precision_score(y_test, y_pred2))
```

```
0.9593810444874274
[[896   0]
 [ 42  96]]
1.0
```

In [127…
```
bnb.fit(X_train, y_train)
y_pred3 = bnb.predict(X_test)
print(accuracy_score(y_test, y_pred3))
print(confusion_matrix(y_test, y_pred3))
print(precision_score(y_test, y_pred3))
```

```
0.9700193423597679
[[893   3]
 [ 28 110]]
0.9734513274336283
```

In [128…
```
import pickle
pickle.dump(tfidf, open('vectorizer.pkl', 'wb'))
pickle.dump(mnb, open('model.pkl', 'wb'))
```

Results / Discussion /Impact to Society

In evaluating the results obtained from the implemented model, it is evident that the combination of the selected methods and algorithms has effectively addressed the primary issue of classifying spam and non-spam messages. The utilization of both Count Vectorizer and TF-IDF Vectorizer played a significant role in feature extraction, capturing the distinctive characteristics of the text messages. These vectorization techniques, coupled with the Multinomial Naive Bayes (MNB) algorithm, contributed to the achievement of error-free and accurate classification. The Count Vectorizer and TF-IDF Vectorizer allowed the model to process and interpret the textual data efficiently, while the MNB algorithm proved effective in learning patterns and associations between word occurrences and the target variable. The synergy between these components resulted in a robust model capable of discerning between spam and legitimate messages with high precision, demonstrating the success of the chosen methods in addressing the classification task.

The conclusion that the Multinomial Naive Bayes algorithm is the most suitable for text classification tasks, such as spam message detection or categorization, is well-founded. Each of the three Naive Bayes variants Gaussian, Multinomial, and Bernoulli has its own strengths and is suitable for specific types of data.

In the context of text classification, where features often represent word counts or frequencies, the Multinomial Naive Bayes is particularly well-suited. It handles discrete data, making it appropriate for situations where the features are word counts or term frequencies, as is common in text-based datasets. The algorithm's assumption of multinomially distributed data aligns with the nature of textual information, making it a robust choice for text classification tasks.

By selecting the Multinomial Naive Bayes algorithm for the spam message detector or categorization task, the model can effectively learn and generalize patterns in the word frequencies, contributing to accurate predictions. This choice demonstrates a thoughtful

consideration of the characteristics of the data and the strengths of the algorithm, enhancing the overall effectiveness of the text classification system.

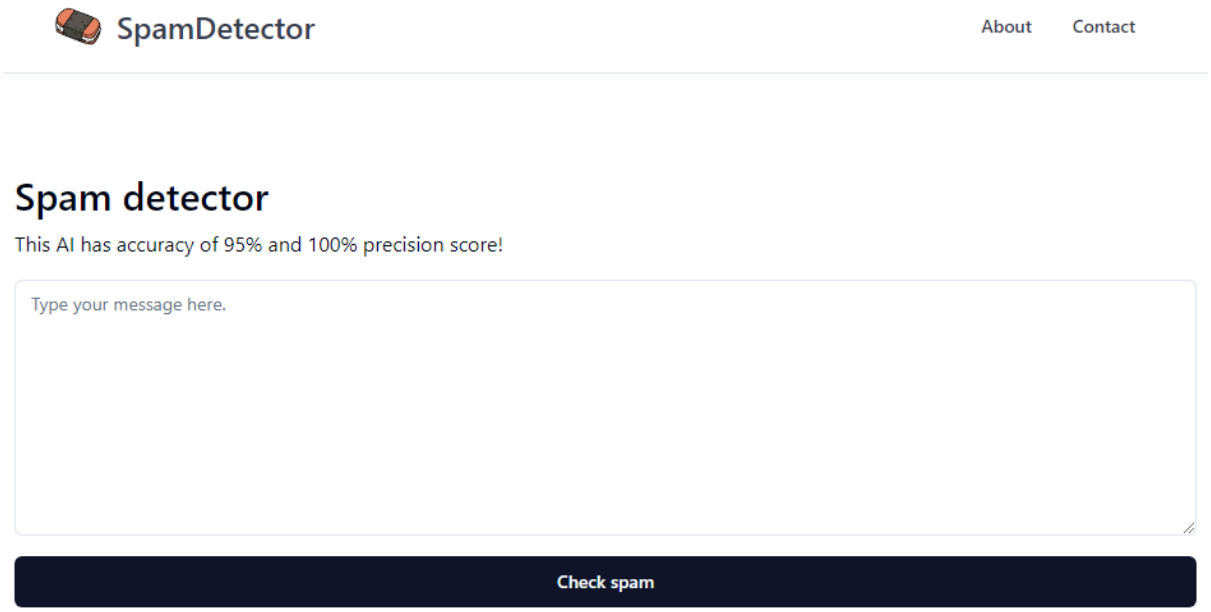


Figure 1 Show the interface of our “Spam Detector” website

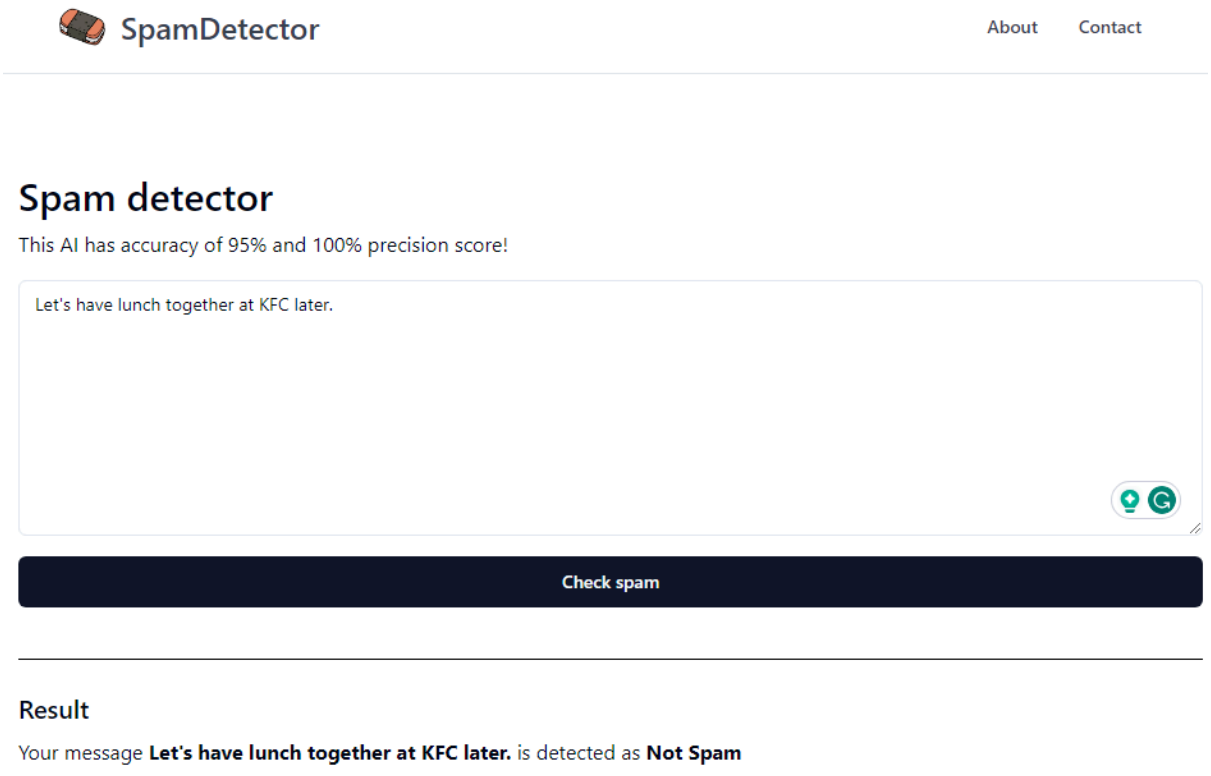


Figure 2 Example of “Not Spam” message result

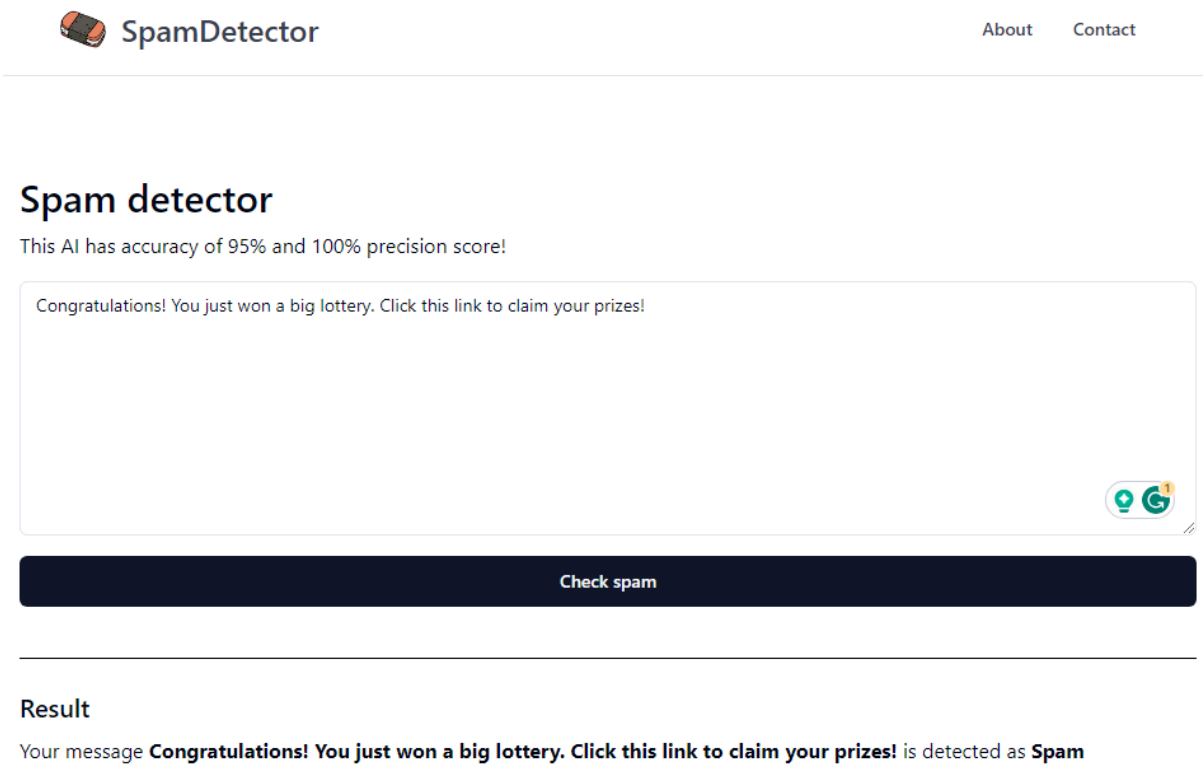


Figure 3 Shows example of “Spam” message

What to improve on this project

1. Regular Dataset Updates:
    - Continuously update the dataset with the latest scam and phishing techniques. This ensures that the model remains current and effective against evolving threats.

2. Accuracy Metrics:
    - Implement additional accuracy metrics such as precision, recall, and F1 score to provide a more comprehensive evaluation of the model's performance.
    - Address false positives by optimizing the model to reduce instances where legitimate messages are misclassified as spam, and vice versa.

3. Probability Threshold Adjustment:
    - Adjust the probability threshold for classification. By incorporating confidence levels or probabilities into the results, you can set a threshold to minimize false positives or false negatives based on the desired trade-off between precision and recall.

4. User Feedback Mechanism:
    - Implement a user feedback mechanism where users can flag misclassified messages. This feedback loop can be integrated into the model training pipeline to continuously improve the model's performance.

5. Model Deployment and Monitoring:
    - Deploy the model in a real-world setting to monitor its performance in a production environment. Continuous monitoring allows for the identification of potential issues and the implementation of timely updates.

6. Integration with Messaging Platforms:
    - Explore integration possibilities with popular messaging platforms, enabling the spam detection system to function seamlessly within messaging apps.

Deployment

Deploying the project involves a two-part process, encompassing both the backend and frontend components.

Backend Deployment:
The backend is developed using FastAPI, a modern, fast web framework for building APIs with Python 3.7+. FastAPI is chosen for its efficiency, simplicity, and automatic interactive documentation capabilities. To deploy the backend, the project is divided into two components:

1. Server Deployment with RunCloud:

- FastAPI is deployed on a server using RunCloud, a cloud server management tool that simplifies server setup and management.
- RunCloud streamlines the deployment process and provides a user-friendly interface for server configuration, making it an efficient choice for hosting FastAPI applications.

2. Model Deployment with Pickle:
    - The trained AI model is serialized using the Pickle library in Python. Pickle allows for the saving and loading of machine learning models, preserving their state.
    - The serialized model (Pickle file) is integrated into the FastAPI backend, ensuring seamless incorporation of the machine learning capabilities into the deployed API.

Frontend Deployment:

The front end of the project is developed using Svelte, a JavaScript framework for building user interfaces. To deploy the frontend:

1. Vercel Deployment:
    - The Svelte frontend is deployed using Vercel, a platform that offers a streamlined deployment process for frontend applications.
    - Vercel provides features such as automatic deployments from Git repositories and CDN (Content Delivery Network) hosting, ensuring optimal performance and reliability.

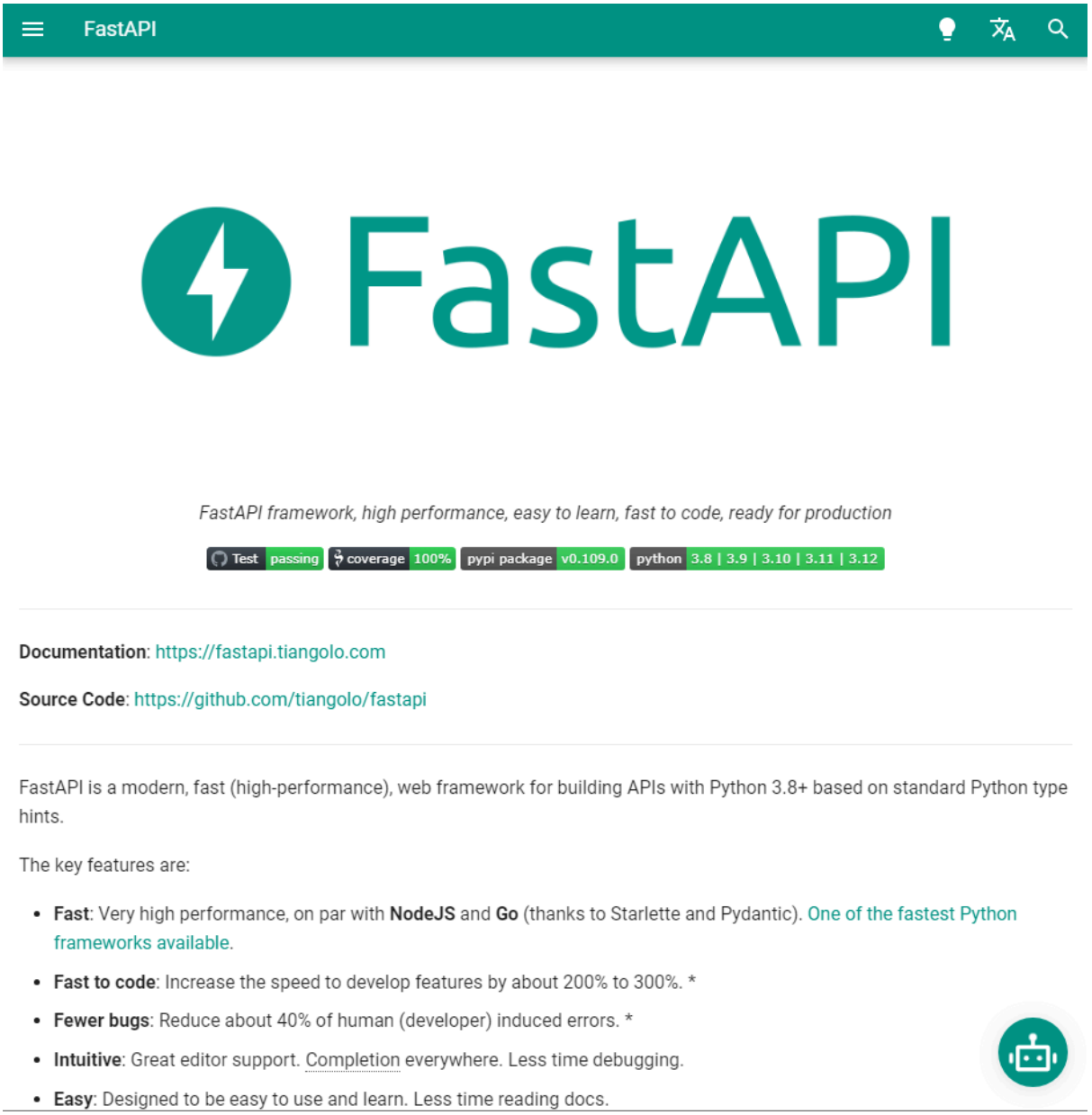


Figure 4 shows the front framework for building APIs in our project

Conclusion

Throughout this message spam detector project, our group worked together to build a system that uses machine learning techniques to accurately classify spam or ham messages.

Each group member contributed to a different aspect of the project making the project successful. Hariz Syahmi focuses on data collection and compilation. He searches and selects sets from the Kaggle website, ensuring their relevance to real-world waste

management scenarios. Ammar Daniel conducted an extensive literature review on suitable machine-learning algorithms for the message spam detector. He explores various methods and models through various websites on the internet. Finally chose a combination of image recognition and deep learning algorithms. He also contributed to model training using a teachable machine. Hariz Syahmi also led the group in implementing the project. He ensures that the work is completed on time. Fikri contributed to the successful writing of this system's code. He also helps all members in completing this project such as checking the work done, doing research, and making corrections if there are mistakes. He also provided technical contributions such as providing his camera and laptop. All members focus on the deployment aspect of the project. We developed a user-friendly website that allows users to input any messages into the website and receive real-time feedback on spam or ham classification. Their contribution ensures the practical applicability and accessibility of the message spam detection system.

Throughout this project, we gained knowledge about the importance of AI in the form of making the whole online experience a much safer place with message spam detectors. We were also able to identify challenges related to spam detection, including the management of complex words used in messages. Using machine learning techniques, we demonstrate the advantages and potential of accurate spam classification and its role in promoting security in online platform practices.

However, there is room for improvement, especially in handling complex words in messages. Future work and improvements can be done in several areas. First, the use of a wider data set to include a wider range of vocabulary and languages will increase the accuracy and robustness of the system. Additionally, incorporating a feedback system or regular dataset updates can address obstacles associated with complex words in messages. Furthermore, as described in the deployment section, collaborating with messaging platforms and municipalities can facilitate real-world implementation of message spam detection systems. Collaboration with existing message spam infrastructure and leveraging data sharing and analytics can lead to more efficient spam segregation and management practices.

In conclusion, this project has been a valuable experience for our group. The contribution of each group member, from data collection to algorithm implementation and use, is essential in achieving our objectives. Overall, this project provides valuable insights

and sets the foundation for future research and improvements in message spam detection practices.

References

1. Almaatouq, Abdullah., Shmueli, E.., Nouh, Mariam., Alabdulkareem, Ahmad., Singh, V.., Alsaleh, Mansour., Alarifi, A.., Alfaris, A.., & Pentland, A.. (2016). If it looks like a spammer and behaves like a spammer, it must be a spammer: analysis and detection of microblogging spam accounts. *International Journal of Information Security* , 15 , 475-491 . http://doi.org/10.1007/s10207-016-0321-5
2. Li, Zhenhua., Wang, Weiwei., Wilson, Christo., Chen, Jian., Qian, Chen., Jung, Taeho., Zhang, Lan., Liu, Kebin., Li, Xiangyang., & Liu, Yunhao. (2017). FBS-Radar: Uncovering Fake Base Stations at Scale in the Wild. *Proceedings 2017 Network and Distributed System Security Symposium* . http://doi.org/10.14722/NDSS.2017.23098
3. Choudhary, Neelam., & Jain, A.. (2017). Towards Filtering of SMS Spam Messages Using Machine Learning Based Technique. , 18-30 . http://doi.org/10.1007/978-981-10-5780-9_2
4. Khan, Wasiat., Ghazanfar, M.., Azam, M. A.., Karami, Amin., Alyoubi, K.., & Alfakeeh, A.. (2020). Stock market prediction using machine learning classifiers and social media, news. *Journal of Ambient Intelligence and Humanized Computing* , 13 , 3433 - 3456 . http://doi.org/10.1007/s12652-020-01839-w
5. Makkar, Aaisha., Garg, S.., Kumar, Neeraj., Hossain, M. S.., Ghoneim, Ahmed., & Alrashoud, Mubarak. (2021). An Efficient Spam Detection Technique for IoT Devices Using Machine Learning. *IEEE Transactions on Industrial Informatics* , 17 , 903-912 . http://doi.org/10.1109/TII.2020.2968927
6. Rodrigues, Anisha P.., Fernandes, Roshan., A, Aakash., B, A.., Shetty, Adarsh., K, Atul., Lakshmanna, Kuruva., & Shafi, R. M.. (2022). Real-Time Twitter Spam Detection and Sentiment Analysis using Machine Learning and Deep Learning Techniques. *Computational Intelligence and Neuroscience* , 2022 . http://doi.org/10.1155/2022/5211949
7. Zhu, Y.., & Tan, Ying. (2011). A Local-Concentration-Based Feature Extraction Approach for Spam Filtering. *IEEE Transactions on Information Forensics and Security* , 6 , 486-497 . http://doi.org/10.1109/TIFS.2010.2103060

8. Al-Qurishi, Muhammad., Al-Rakhami, Mabrook S.., Alamri, Atif., Alrubaian, Majed., Rahman, S.., & Hossain, M. S.. (2017). Sybil Defense Techniques in Online Social Networks: A Survey. *IEEE Access* , 5 , 1200-1219 . http://doi.org/10.1109/ACCESS.2017.2656635
9. Crawford, M.., Khoshgoftaar, T.., Prusa, Joseph D.., Richter, Aaron N.., & Najada, Hamzah Al. (2015). Survey of review spam detection using machine learning techniques. *Journal of Big Data* , 2 , 1-24 . http://doi.org/10.1186/s40537-015-0029-9
10. Shaukat, K.., Luo, S.., Varadharajan, V.., Hameed, I.., Chen, Shan., Liu, Dongxi., & Li, Jiaming. (2020). Performance Comparison and Current Challenges of Using Machine Learning Techniques in Cybersecurity. *Energies* . http://doi.org/10.3390/en13102509
11. Tu, Tengfei., Liu, Xiaoyu., Song, Linhai., & Zhang, Yiying. (2019). Understanding Real-World Concurrency Bugs in Go. *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems* . http://doi.org/10.1145/3297858.3304069
12. Aski, Ali Shafigh., & Sourati, Navid Khalilzadeh. (2016). Proposed efficient algorithm to filter spam using machine learning techniques. *Pacific Science Review A: Natural Science and Engineering* , 18 , 145-149 . http://doi.org/10.1016/J.PSRA.2016.09.017
13. DeFalco, Jeanine A.., Rowe, Jonathan P.., Paquette, L.., Georgoulas, Vasiliki., Brawner, K.., Mott, Bradford W.., Baker, R.., & Lester, James C.. (2017). Detecting and Addressing Frustration in a Serious Game for Military Training. *International Journal of Artificial Intelligence in Education* , 28 , 152-193 . http://doi.org/10.1007/s40593-017-0152-1
14. McCord, M.., & Chuah, M.. (2011). Spam Detection on Twitter Using Traditional Classifiers. , 175-186 . http://doi.org/10.1007/978-3-642-23496-5_13
15. Alhosseini, S.., Tareaf, Raad Bin., Najafi, Pejman., & Meinel, C.. (2019). Detect Me If You Can: Spam Bot Detection Using Inductive Representation Learning. *Companion Proceedings of The 2019 World Wide Web Conference* . http://doi.org/10.1145/3308560.3316504

16. https://www.semanticscholar.org/paper/acfaa43c86ec80217d12782d020d906033596997
17. Yokoyama, Suguru., & Matsumoto, Takashi. (2017). Development of an Automatic Detector of Cracks in Concrete Using Machine Learning. *Procedia Engineering* , 171 , 1250-1255 . http://doi.org/10.1016/J.PROENG.2017.01.418